

포인터

컴퓨터학부 박한영



목차

01

포인터

02

함수 포인터

03

동적 할당



포인터 개념

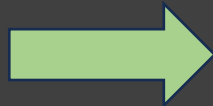
- 포인터는 메모리 주소를 저장하는 변수
- 일반 변수는 저장된 데이터의 값을 제공하지만, 포인터는 데이터의 위치(주소)를 제공함!
- 포인터는 메모리를 직접 다룬다는 점에서 C언어의 핵심 기능
- 포인터 변수의 크기는 운영체제에 따라 달라짐(32bit: 4bytes, 64bit: 8bytes)
- 참조할 변수의 자료형에 따라 포인터가 읽는 메모리 크기가 달라짐
 - 예: `double *ptr;` 의 경우 8바이트만큼 메모리를 가져옴



포인터 선언

- 포인터는 *(asterisk) 연산자를 이용해 선언
- `int *ptr;` 이라고 선언했다고 가정해보자
 - `ptr`은 '`ptr`이 가리키는 것의 주소(address)' 를 의미함
 - `*ptr`은 '`p`가 가리키는 것(value)' 을 의미함
 - `int`는 `*ptr`의 자료형을 나타냄

```
int Var = 3;
int *ptr = &Var;
printf("The value of Var: %d\n", Var);
printf("The address of Var: %#x\n", &Var);
printf("The reference of ptr: %d\n", *ptr);
printf("The value of ptr: %#x", ptr);
```

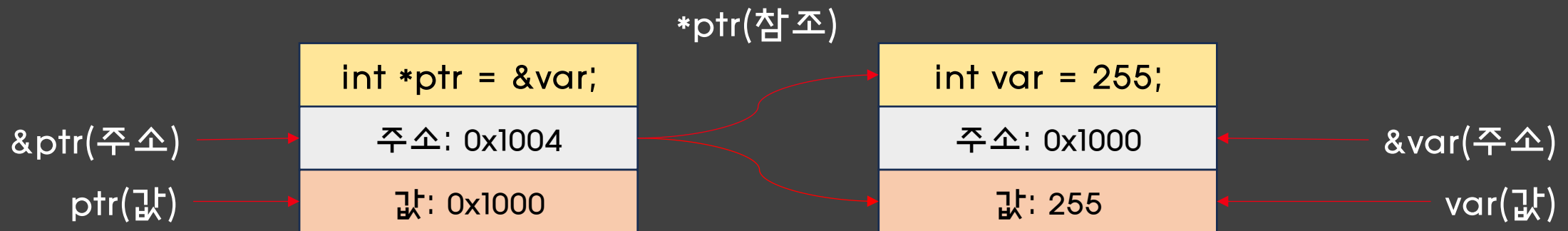


```
The value of Var: 3
The address of Var: 0x3e1ffa64
The reference of ptr: 3
The value of ptr: 0x3e1ffa64
```



주소와 참조

- &(주소 연산자)는 해당 변수의 주소 값을 반환
- *(참조 연산자)는 해당 변수를 참조한 값을 반환
- 포인터 변수에 *을 붙이는 건 변수에 저장된 주소에 저장된 값을 본다는 의미



call by value(복습)

```
int main() {  
    int a, b;  
    sum(a, b);  
}
```

함수 호출

```
int sum(int x, int y) {  
    return x + y;  
}
```

a, b가 x, y로 복사



- 복사의 의미
- a와 b를 함수 sum의 매개변수 자리에 넣음
- 이 변수를 sum에서 사용하는 것이 아니라, a와 b에서 저장된 값만 복사해서 사용



call by address(복습)

```
int main() {  
    int a, b;  
    sum(&a, &b);  
}
```

함수 호출

a, b의 주소가 x, y로 전달

```
int sum(int *x, int *y) {  
    return *x + *y;  
}
```

- 변수의 주소를 전달
- a와 b의 주소를 함수 sum의 매개변수로 전달
- 원본 변수에 직접 접근할 수 있으므로, 함수 안에서 값을 변경할 수 있음
- 매개변수의 크기가 큰 경우 주소를 전달하는 방식이 훨씬 효율적임



포인터 사용 시 주의사항

- 포인터는 반드시 초기화 후 사용해야 안전함! (NULL을 이용)
- *(참조 연산자)와 &(주소 연산자)는 역함수 관계를 가짐!
 - 예: int n; 일 때 $&(*n) == n$, $&(*(*n)) == *n$ 과 같음
 - 참고: C99 표준에서는 $\&\&n$ 과 같은 표현은 허용하지 않음

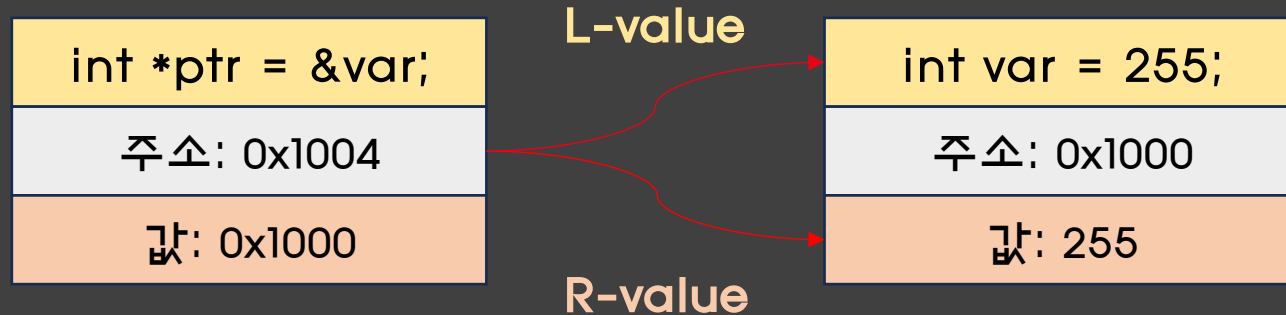
```
// 초기화되지 않은 포인터
int *ptr;
// 알 수 없는 주소에 접근: 위험!
*ptr = 10;
// NULL로 초기화하기
ptr = NULL;
int n = 10;
int *p = &n; // *p == n
int **pp = &p; // **pp == n
int ***ppp = &pp; // ***ppp == n
```



포인터 사용 시 주의사항

- 포인터는 참조 시 L-value / R-value에 따라 가리키는 대상이 달라짐!
- L-value(식별자)인 경우 변수 자체(=원본)를 의미함
- R-value(참조자)인 경우 변수의 값을 의미함

```
int Var = 3, tmp;  
int *ptr = &Var;  
// Case of L-value  
// Var = 5 와 같음!!  
*ptr = 5;  
// Case of R-value  
// tmp = 2 + 5 와 같음!!  
tmp = 2 + *ptr;
```



연산자(복습)

연산자 우선 순위

기호 1	연산 유형	associativity
[] () . -->+++ (후위)	식	왼쪽에서 오른쪽
sizeof & * + - ~ !+--- (전위)	단항	오른쪽에서 왼쪽
형식 캐스팅	단항	오른쪽에서 왼쪽
* / %	곱하기	왼쪽에서 오른쪽
+ -	더하기	왼쪽에서 오른쪽
<< >>	비트 시프트	왼쪽에서 오른쪽
< > <= >=	관계	왼쪽에서 오른쪽
== !=	같음	왼쪽에서 오른쪽
&	비트 AND	왼쪽에서 오른쪽
^	비트 제외 OR	왼쪽에서 오른쪽
	비트 포함 OR	왼쪽에서 오른쪽
&&	논리 AND	왼쪽에서 오른쪽
	논리 OR	왼쪽에서 오른쪽
? :	조건식	오른쪽에서 왼쪽
= *= /= %+= -= <<= >>= &^= =	단순 및 복합 할당 2	오른쪽에서 왼쪽
,	순차적 계산	왼쪽에서 오른쪽



포인터 연산

- 포인터 변수에 증감 연산을 적용하면 주소를 이동하면서 각 요소에 접근할 수 있음
- `ptr + 1`은 주소값이 1만큼 증가하는 것이 아닌, 자료형의 크기만큼 이동함
- 증감연산자가 참조연산자보다 우선순위가 높으므로 계산 시 유의하며, 헛갈리는 경우에는 반드시 괄호를 통해 연산을 구분할 것!

```
int arr[5] = {1,2,3,4,5};  
for(int i = 0; i < 5; i++)  
// arr는 배열의 시작 주소임을 기억!  
// 배열의 시작점에서 i만큼 이동  
|   printf("%d\n", *(arr + i));
```

```
int *ptr;  
// ptr++ 먼저 진행 후  
// ptr을 참조하는 식  
*ptr++;  
// 참조값을 증가시키기  
(*ptr)++;
```



포인터와 배열

- 배열은 주소값이 고정돼있는 상수 포인터라고도 볼 수 있음 (즉, L-value로 사용 x)
- 반면 포인터는 변수이므로 L-value로 사용할 수 있음
- 포인터가 배열을 가리키는 경우 기존과 동일하게 배열의 원소에 접근 가능

```
// const int *arr 와 유사함
// 주소를 수정할 수 없음
int arr[3] = {1,2,3};
int *ptr;
arr = ptr; // Error!
// arr[i]와 동일하게 접근 가능
for(int i = 0; i < 3; i++)
|   printf("%d\n", ptr[i]);
```



포인터와 문자열

- 문자열을 포인터로 선언하는 경우 상수와 같이 취급함
 - 예: `char *str = "Jaram"` ; , 이 경우 문자열은 코드 영역에 저장
- 문자열의 배열이 필요한 경우, 이중포인터를 사용하여 구현해야 함

```
int n;
scanf("%d", &n);
// char* 를 n개 저장할 배열
char **arr = (char **)malloc(sizeof(char *) * n);
// 각 문자열 공간 할당
for (int i = 0; i < n; i++)
    arr[i] = (char *)malloc(sizeof(char) * 100);
// 문자열 입력
for (int i = 0; i < n; i++)
    scanf("%s", arr[i]);
// 출력
for (int i = 0; i < n; i++)
    printf("%s\n", arr[i]);
// 메모리 해제
for (int i = 0; i < n; i++)
    free(arr[i]);
free(arr);
```



void 포인터

- C에서의 void는 보통 아무것도 없음을 의미함
- 그러나 void*는 특정 자료형이 정해지지 않은 포인터로서, 어떤 자료형의 주소든지 저장 가능함.
- 그러나 void 포인터는 바로 값을 참조할 수 없으므로, 반드시 명시적 형변환 후 값을 참조하여야 함
- 뒤에서 다룰 malloc이나 free 같은 함수들이 void* 타입을 리턴

```
void* ptr;  
int a = 3;  
float pi = 3.14;  
// 자료형과 상관없이 주소 저장  
ptr = &a;  
ptr = &pi;  
// type을 몰라 err 발생!  
float tmp = *ptr;
```

함수 포인터

- 함수도 다른 변수들처럼 메모리에 저장되고 시작 주소를 가짐
- 함수 포인터는 함수의 주소를 저장하는 포인터
- 함수를 변수처럼 저장하고 호출할 수 있음

```
int add(int, int);

int main() {
    // 함수 포인터
    // (자료형) (변수명)(매개변수);
    int (*fp)(int, int);
    // 포인터에 함수 주소 배정
    fp = add;
    // 함수 포인터 사용
    printf("%d", fp(2,3));
}
```



메모리 영역(복습)

스택 영역

- 지역 변수와 함수의 매개 변수가 저장되는 영역
- 스택 영역은 함수의 호출과 함께 할당되며, 함수의 호출이 완료되면 소멸됨
- 스택 영역에 저장되는 함수의 호출 정보를 스택 프레임이라고 함

힙 영역

- 사용자가 직접 관리하는 메모리 영역
- 동적 메모리가 이곳에서 할당되고 해제됨
- 메모리 상위 주소에서 하위 주소의 방향으로 할당



동적 할당

- 동적할당을 사용하면 프로그램을 실행하는 동안 필요한만큼 공간을 할당 받을 수 있음
- C언어에서는 `*malloc(size_t)` 함수를 사용해 메모리를 할당함

```
1 int *arr; // 포인터 변수 선언
2 arr = malloc(sizeof(int) * 5); // int 5개 만큼 동적 할당
```

Stack 영역

(자동 저장 영역)

arr 0x1000

포인터 변수 arr 는
Heap 영역의 시작 주소
0x1000 을 저장한다.

Heap 영역

(동적 할당 영역)

int 형 5개 (각 4Byte)

arr[0]	arr[1]	arr[2]	arr[3]	arr[4]
0x1000	0x1004	0x1008	0x100C	0x1010

인덱스	0	1	2	3	4
주소	0x1000	0x1004	0x1008	0x100C	0x1010

- `malloc(sizeof(int) * 5)` 는 int 크기(4Byte) * 5 만큼의 연속된 메모리를 Heap 영역에 할당한다.
- arr 은 그 시작 주소(여기서는 0x1000)를 가리킨다.

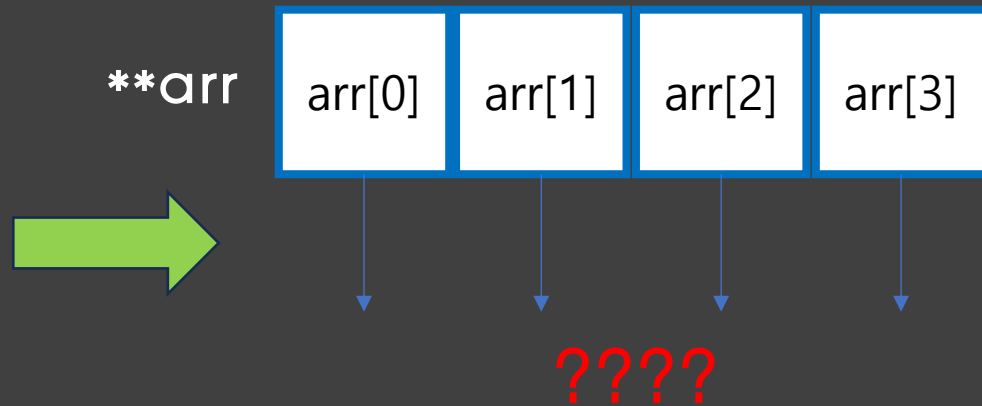
사용 후
`free(arr);` ⇒ Heap 메모리 반환



동적 할당

- 이중 포인터를 사용하는 경우, 동적 할당을 두 번 진행해야 함
- 만약 한 번만 하는 경우는 배열 요소들의 주소를 저장할 공간만 만들어지고, 각 요소들에 대한 공간은 할당되지 않음
- 해제도 동일하게 각 요소들에 대해 해제해주어야 함

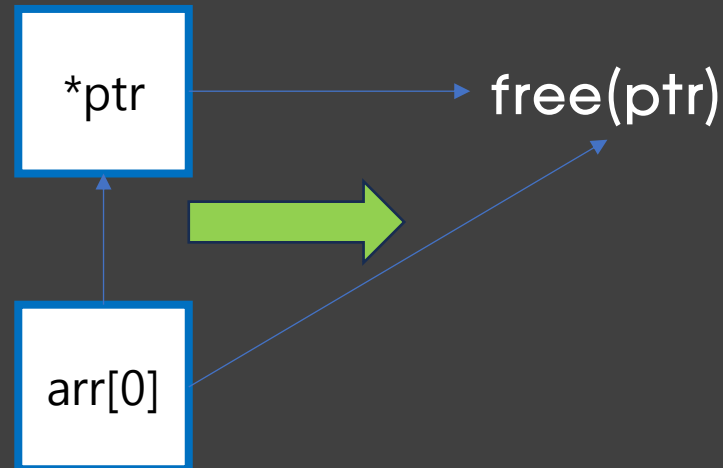
```
// 이중 포인터
int **arr;
arr = malloc(sizeof(int*) * 4);
// 만약 나머지 malloc을 안하면?
// for(int i = 0; i < 4; i++)
//     arr[i] = malloc(sizeof(int) * 5);
```



동적 할당

- 동적 메모리를 모두 사용한 후에는 반드시 이를 반납해야 함
- `free()` 함수를 통해 메모리를 해제할 수 있음
- 사용하지 않는 메모리를 반납하지 않는 경우 메모리 누수가 일어남
- dangling pointer: 어떤 포인터가 해제된 메모리를 가리키는 경우를 말함

```
int *ptr;  
// 동적 할당  
ptr = malloc(sizeof(int) * 3);  
// 다른 포인터  
int *elem = ptr + 1;  
// 메모리 해제  
free(ptr);  
// 나는 어디?  
// 해제된 메모리를 가리킴!!!  
// = dangling pointer  
*elem = 3;
```



THANK YOU!

